

AULA 11

O mundo, a deriva e o mapa que corrige

Etapa 6 · terça, 29/09/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

A odometria não erra por ruído. Erra por viés.

Ruído se cancela: medir mais vezes ajuda. Viés não — ele entra na conta sempre com o mesmo sinal, e cresce com a distância percorrida.

Três gates do TP3 começam nesta semana

- G3.0 · bringup.launch.py sobe o sistema inteiro · vence AMANHÃ, 30/09
- G3.1 · mundo de simulação com robô e câmera · vence 06/10
- G3.2 · árvore map -> odom -> base_link -> sensores · vence 10/10
- O G3.2 é o gate de verdade do TP3 — o resto do trabalho se apoia nele

A aula está ordenada por custo de perda, como a anterior

- 1. Launch — G3.0, vence amanhã — não pode cair
- 2. O mundo como produtor de dados — G3.1, 06/10 — não pode cair
- 3. map -> odom e a deriva — G3.2, 10/10 — não pode cair
- 4. O que o Nav2 acrescenta — TP4, 30/10 — volta na Aula 12 se faltar tempo

Semana passada o corte funcionou como combinado.

O bloco de launch era o de folga, foi avisado no primeiro minuto, e foi ele que cedeu. Ele abre hoje, a tempo do gate. O que se perde por decisão custa menos que o que se perde por acidente.

PARTE 1

Launch: um comando sobe o sistema inteiro

Três mecanismos, e os três reaparecem no seu bringup

- Compor — IncludeLaunchDescription traz o subsistema inteiro, com seus argumentos
- Parametrizar — o YAML vem de fora, então o mesmo launch sobe teste e bancada
- Condicionar — IfCondition liga e desliga pedaços
- O bringup não sabe o que cada nó faz: conhece nome e parâmetros. Contrato, de novo

O exemplo sobe um sistema pequeno e completo

```
ros2 launch aula10_bringup bringup.launch.py  
ros2 launch aula10_bringup bringup.launch.py modelo:=false
```

Sem câmera e sem hardware.



Um launch que sobe sem erro não é um launch que funciona.

Subir é o que o launch faz. Funcionar é outra afirmação — e ela precisa de prova.

As três verificações

```
ros2 launch aula10_bringup bringup.launch.py --show-args # o que dá para configurar
ros2 node list                                           # quem realmente subiu
ros2 param dump /percepcao/detector                    # com que valores
```

A terceira é a que quase ninguém faz. É ela que pega o erro do próximo slide.

O YAML que ninguém lê

```
ros2 launch aula10_bringup bringup.launch.py \  
  params:=../config/sistema-armadilha.yaml  
  
ros2 param get /percepcao/detector taxa_hz  
-> 2.0          # o padrão do código, não o 9.0 do arquivo
```

O arquivo pedia 9.0. O nó subiu com 2.0. Nenhum erro foi impresso em lugar nenhum.

A causa: a chave do YAML não casa com o nome do nó

- detector: significa o nó /detector, na raiz
- Mas o launch empurrou o nó para /percepcao/detector
- A regra do ROS 2 é ignorar o que não casa — sem avisar
- Curas: /percepcao/detector: ou /**: — prefira o curinga
- Ele sobrevive à mudança de namespace, que é o que você vai fazer quando o sistema crescer

É a mesma família de falhas que vocês já conhecem

- O `view_frames` que gera PDF vazio quando o launch não está rodando
- A saturação que vira ruído puro quando o valor é quase zero
- O `ros2 action list` que funciona sem `source` — e o `send_goal` que não
- São erros que não se apresentam como erro
- A defesa: verificar a afirmação, em vez de tratar o silêncio como aprovação

Sobre ordem: nada espera nada

- O supervisor quase sempre nasce antes do detector, reclama, e se recupera sozinho
- Isso é o comportamento correto, não defeito
- Em ROS 2 o grafo é descoberto, não montado em ordem
- Nó que exige ordem de subida quebra no dia em que a máquina está lenta
- A regra: o nó tolera a ausência do outro

PARTE 2

O mundo é um produtor de dados, não uma janela

O TP3 pede um cenário. Do que o seu sistema realmente depende?

- Não do Gazebo
- De um tópico /scan
- De um tópico /odom
- De uma árvore de TF que ligue os dois ao robô
- Qualquer coisa que produza os três serve — e é por isso que o exemplo de hoje existe

O mundo mínimo: sem física, sem render, sem janela

```
ros2 launch aula11_mundo mundo.launch.py  
ros2 topic hz /scan  
ros2 topic echo /odom --once
```

12 x 8 metros, duas salas ligadas por uma porta, três obstáculos. Laser por ray-casting sobre grade de ocupação, em numpy.



Isto não é desistir do Gazebo. É chegar nele com o esqueleto pronto.

Troque o nó mundo por um Gazebo publicando nos mesmos tópicos e todo o resto continua valendo: o launch, os frames, o SLAM, o Nav2.

Quem publica cada aresta — e saber isto é metade do G3.2

- `base_footprint` -> `camera_link` · `robot_state_publisher`, lendo o seu URDF (foi o G2.5)
- `base_footprint` -> `laser_frame` · transformada estática: o sensor não se mexe
- `odom` -> `base_footprint` · o nó mundo, integrando a odometria
- `map` -> `odom` · hoje, identidade fixa — um PLACEHOLDER
- E a aresta de cima de tudo está, neste momento, mentindo

PARTE 3

map -> odom: a aresta que corrige

Veja o número crescer

```
ros2 launch aula11_mundo mundo.launch.py
ros2 topic echo /deriva          # noutro terminal

ros2 launch aula11_mundo mundo.launch.py deriva:=0.0    # e compare
```

/deriva é a distância entre onde o robô ESTÁ e onde a odometria ACHA que ele está.

O modelo erra de um jeito específico, e o jeito importa

- Erro de escala de 3% — o de quem mediu o raio da roda com régua
- Não é aleatório: tem sinal constante
- Ruído você combate com repetição, porque ele se cancela
- Foi por isso que a Aula 9 usou mediana em vez de uma medida só
- Viés não se cancela nunca: cresce com a distância percorrida. Rodar mais tempo piora

Nenhum filtro sobre /odom resolve.

O conserto não está em medir melhor a roda. Está em medir contra outra coisa — algo que não se mexe.

É isso que o SLAM faz, e é literalmente o trabalho dele

- Compara o /scan de agora com o mapa que vem construindo
- Calcula quanto a odometria já errou
- Publica essa correção como map -> odom
- Enquanto essa aresta for identidade, o robô acredita na própria odometria
- E no RViz2 você vê o laser deslizar para fora das paredes

Noventa por cento dos problemas de SLAM e de Nav2 são problemas de TF disfarçados.

Se o `view_frames` mostrar árvore quebrada ou dois `odom`, pare tudo e conserte antes de tocar no G3.3. Insistir no SLAM com TF errada é queimar uma semana.

PARTE 4

O que o Nav2 acrescenta

O piloto do exemplo é o navegador mais burro que funciona

- Anda reto. Gira quando fecha na frente. Trinta linhas
- E explora o mundo inteiro sozinho
- Repare no que ele NÃO tem — é essa lista, e não outra coisa, que o Nav2 é

Cada falta do piloto vira um pedaço do Nav2

- Não sabe onde está -> localização (AMCL) contra o mapa
- Não sabe para onde vai -> um objetivo, enviado como action
- Não vê além do laser -> custos global e local, construídos do mapa
- Não se recupera ao travar -> comportamentos de recuperação

O objetivo do Nav2 é uma action.

Com feedback periódico e cancelamento — exatamente as da Aula 8. O Nav2 não é assunto novo: é a montagem de peças que já estão no lugar.

Tarefa da semana

- G3.0 · seu bringup.launch.py subindo tudo · evidência: `ros2 node list` · vence AMANHÃ
- G3.1 · seu cenário com robô e câmera · vence 06/10
- G3.2 · árvore map -> odom -> base_link completa · frames.pdf · vence 10/10
- Gazebo não roda na sua máquina? O mundo mínimo conta — e você documenta a escolha

Para a próxima aula, 06/10 — SLAM Toolbox

- A aresta map -> odom deixa de ser identidade e passa a ser calculada
- O mapa do G3.3 sai disso
- Chegue com o G3.0 entregue e a árvore do G3.2 fechada
- O SLAM é construído sobre a TF — ele não conserta TF errada